

‘Home work problems -- Data structures’

1. Define an integer array. Declare an integer pointer and initialize it to the first element of the array. Use pointer arithmetic to traverse the array and print the elements.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[5] = {10, 20, 30, 40, 50}; 5 int *p; 6 int i; 7 p = a; 8 printf("Array elements:\n"); 9 for(i = 0; i < 5; i++) 10 { 11 printf("%d ", *p); 12 p++; 13 } 14 return 0; 15 } 16</pre>		<pre>Array elements: 10 20 30 40 50 === Code Execution Successful ===</pre>

2. Modify the elements of an array using pointer arithmetic.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[5] = {10, 20, 30, 40, 50}; 5 int *p; 6 int i; 7 8 p = a; 9 10 for(i = 0; i < 5; i++) 11 { 12 *p = *p + 5; 13 p++; 14 } 15 16 printf("Modified array:\n"); 17 18 p = a; 19 20 for(i = 0; i < 5; i++) 21 { 22 printf("%d ", *p); 23 p++; 24 } 25 26 return 0; 27 } 28</pre>		<pre>Modified array: 15 25 35 45 55 === Code Execution Successful ===</pre>

3. Write a C program to define a function that calculates both the sum and product of two numbers. Return these two results to the main function using pointer parameters passed as arguments.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 void calculate(int a, int b, int *sum, int *product) 4 { 5 *sum = a + b; 6 *product = a * b; 7 } 8 9 int main() 10 { 11 int a, b; 12 int sum, product; 13 14 printf("Enter two numbers: "); 15 scanf("%d %d", &a, &b); 16 17 calculate(a, b, &sum, &product); 18 19 printf("Sum = %d\n", sum); 20 printf("Product = %d\n", product); 21 22 return 0; 23 } 24</pre>		<pre>Enter two numbers: 11 31 Sum = 42 Product = 341 === Code Execution Successful ===</pre>

4. Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify records.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 extern void *malloc(unsigned int); 4 extern void free(void *); 5 6 struct Student 7 { 8 int id; 9 char name[20]; 10 float mark; 11 }; 12 13 int main() 14 { 15 struct Student *p; 16 int n, i; 17 18 printf("Enter number of students: "); 19 scanf("%d", &n); 20 21 p = (struct Student *)malloc(n * sizeof(struct Student)); 22 23 for(i = 0; i < n; i++) 24 { 25 printf("Enter ID, Name and Mark for student %d:\n", i + 1); 26 scanf("%d %s %f", 27 &(p + i)->id, 28 (p + i)->name, 29 &(p + i)->mark); 30 } 31 32 printf("\nStudent Details:\n"); 33 34 for(i = 0; i < n; i++) 35 { 36 printf("%2d: %d Name: %s Mark: %.2f\n", 37 i + 1, 38 (p + i)->id, 39 (p + i)->name, 40 (p + i)->mark); 41 } 42 43 free(p); 44 45 return 0; 46 } 47</pre>		<pre>Enter two numbers: 11 31 Sum = 42 Product = 341 === Code Execution Successful ===</pre>

5. Define two functions with the same signature. Declare a function pointer and use the pointer to call both functions.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int add(int a, int b) 4 { 5 return a + b; 6 } 7 8 int multiply(int a, int b) 9 { 10 return a * b; 11 } 12 13 int main() 14 { 15 int (*fp)(int, int); 16 int a = 10, b = 5; 17 18 fp = add; 19 printf("Addition = %d\n", fp(a, b)); 20 21 fp = multiply; 22 printf("Multiplication = %d\n", fp(a, b)); 23 24 return 0; 25 } 26</pre>		<pre>Addition = 15 Multiplication = 50 === Code Execution Successful ===</pre>

6. Declare an integer, an integer pointer and a double pointer. Print the values and addresses at each level.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int x = 25; 6 int *p; 7 int **q; 8 9 p = &x; 10 q = &p; 11 12 printf("Value of x = %d\n", x); 13 printf("Value using pointer = %d\n", *p); 14 printf("Value using double pointer = %d\n", **q); 15 16 printf("\nAddress of x = %p\n", (void *)&x); 17 printf("Address stored in p = %p\n", (void *)p); 18 printf("Address of p = %p\n", (void *)&p); 19 printf("Address stored in q = %p\n", (void *)q); 20 21 return 0; 22 } 23</pre>		<pre>Value of x = 25 Value using pointer = 25 Value using double pointer = 25 Address of x = 0x7ffc012e9cf4 Address stored in p = 0x7ffc012e9cf4 Address of p = 0x7ffc012e9ce8 Address stored in q = 0x7ffc012e9ce8 === Code Execution Successful ===</pre>

7. Sort an array of floating-point numbers using Selection Sort.

```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     float a[5] = {5.5, 2.2, 8.8, 1.1, 4.4};
6     float temp;
7     int i, j, min;
8
9     for(i = 0; i < 4; i++)
10    {
11        min = i;
12
13        for(j = i + 1; j < 5; j++)
14        {
15            if(a[j] < a[min])
16            {
17                min = j;
18            }
19        }
20
21        temp = a[i];
22        a[i] = a[min];
23        a[min] = temp;
24    }
25    printf("Sorted array:\n");
26
27    for(i = 0; i < 5; i++)
28    {
29        printf("%.1f ", a[i]);
30    }
31
32    return 0;
33 }
34
```

Sorted array:
1.1 2.2 4.4 5.5 8.8

=== Code Execution Successful ===

8. Define a structure for a student with ID, name and grade. Create an array of student structures and sort them based on different fields.

```
main.c Run Output
1 #include <stdio.h>
2
3 struct Student
4 {
5     int id;
6     char name[20];
7     float grade;
8 };
9
10 int main()
11 {
12     struct Student s[5], temp;
13     int i, j, choice;
14     printf("Enter details of 5 students:\n");
15     for(i = 0; i < 5; i++)
16     {
17         printf("Student %d\n", i + 1);
18         printf("Enter ID: ");
19         scanf("%d", &s[i].id);
20         printf("Enter Name: ");
21         scanf("%s", s[i].name);
22         printf("Enter Grade: ");
23         scanf("%f", &s[i].grade);
24     }
25     printf("\nSort based on:\n");
26     printf("1. ID\n");
27     printf("2. Grade\n");
28     printf("Enter choice: ");
29     scanf("%d", &choice);
30
31     for(i = 0; i < 4; i++)
32     {
33         for(j = i + 1; j < 5; j++)
34         {
35             if((choice == 1 && s[i].id > s[j].id) ||
36                 (choice == 2 && s[i].grade > s[j].grade))
37             {
38                 temp = s[i];
39                 s[i] = s[j];
40                 s[j] = temp;
41             }
42         }
43     }
44     printf("\nSorted Student Details:\n");
45     for(i = 0; i < 5; i++)
46     {
47         printf("ID: %d Name: %s Grade: %.2f\n",
48             s[i].id,
49             s[i].name,
50             s[i].grade);
51     }
52
53     return 0;
54 }
55
```

Enter details of 5 students:

Student 1
Enter ID: 11
Enter Name: Farah
Enter Grade: 95

Student 2
Enter ID: 31
Enter Name: Harish
Enter Grade: 98

Student 3
Enter ID: 16
Enter Name: Ajay
Enter Grade: 96

Student 4
Enter ID: 21
Enter Name: Isha
Enter Grade: 96

Student 5
Enter ID: 44
Enter Name: Darshini
Enter Grade: 96

Sort based on:
1. ID
2. Grade
Enter choice: 1

Sorted Student Details:
ID: 11 Name: Farah Grade: 95.00
ID: 16 Name: Ajay Grade: 96.00
ID: 21 Name: Isha Grade: 96.00
ID: 31 Name: Harish Grade: 98.00
ID: 44 Name: Darshini Grade: 96.00

=== Code Execution Successful ===

9. Sort a set of negative numbers using Radix Sort.

```
main.c Run Output
1 #include <stdio.h>
2
3 int getMax(int a[], int n)
4 {
5     int max = a[0];
6     int i;
7     for(i = 1; i < n; i++)
8     {
9         if(a[i] > max)
10             max = a[i];
11     }
12     return max;
13 }
14
15 void radixSort(int a[], int n)
16 {
17     int output[10];
18     int count[10];
19     int max, exp = 1;
20     int i;
21
22     max = getMax(a, n);
23
24     while(max / exp > 0)
25     {
26         for(i = 0; i < 10; i++)
27             count[i] = 0;
28
29         for(i = 0; i < n; i++)
30             count[(a[i] / exp) % 10]++;
31
32         for(i = 1; i < 10; i++)
33             count[i] = count[i] + count[i - 1];
34
35         for(i = n - 1; i >= 0; i--)
36         {
37             output[count[(a[i] / exp) % 10] - 1] = a[i];
38             count[(a[i] / exp) % 10]--;
39         }
40
41         for(i = 0; i < n; i++)
42             a[i] = output[i];
43
44         exp = exp * 10;
45     }
46 }
47
48 int main()
49 {
50     int a[] = {-45, -12, -89, -34, -7, -56};
51     int b[];
52     int i;
53
54     for(i = 0; i < 6; i++)
55         b[i] = -a[i];
56
57     radixSort(b, 6);
58 }
```

Sorted negative numbers:
-89 -56 -45 -34 -12 -7

--- Code Execution Successful ---

10. Write a C program to copy one array to another array using pointers.

```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[5], b[5];
6     int *p, *q;
7     int i;
8
9     printf("Enter 5 elements:\n");
10
11     for(i = 0; i < 5; i++)
12     {
13         scanf("%d", &a[i]);
14     }
15
16     p = a;
17     q = b;
18
19     for(i = 0; i < 5; i++)
20     {
21         *q = *p;
22         p++;
23         q++;
24     }
25
26     printf("Original array:\n");
27
28     p = a;
29
30     for(i = 0; i < 5; i++)
31     {
32         printf("%d ", *p);
33         p++;
34     }
35
36     printf("\nCopied array:\n");
37
38     q = b;
39
40     for(i = 0; i < 5; i++)
41     {
42         printf("%d ", *q);
43         q++;
44     }
45
46     return 0;
47 }
```

Enter 5 elements:
2
3
4
5
6

Original array:
2 3 4 5 6

Copied array:
2 3 4 5 6

--- Code Execution Successful ---